

JAVA COLLECTION FRAMEWORK

Important Methods by Interface – Task 5

Experiment No.	05
Title	Java Collection Framework: Important Methods by Interface
Source File	CollectionMethodsDemo.java
Package	javacore

Objective / Aim

To study and demonstrate the important methods of the Java Collection Framework using Collection, List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator.

System Requirements

Hardware

- Standard PC/Laptop
- Intel Core i3/i5 processor or equivalent
- Minimum 4 GB RAM
- Minimum 500 MB free storage

Software

- Windows / Linux / macOS
- JDK 17 or JDK 21
- Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Important Eclipse Note

The corrected code includes the required class declaration. The first lines must be:

```
package javacore;  
import java.util.*;  
public class CollectionMethodsDemo {
```

The main() method is inside this class. Save the file as **CollectionMethodsDemo.java**.

Algorithm / Logic Description

1. Import java.util.* to access collection interfaces and classes.
2. Create a List and demonstrate adding, accessing, replacing, removing, searching, and sorting elements.
3. Create a Set and demonstrate unique elements.
4. Create SortedSet and NavigableSet objects and demonstrate sorted and navigation operations.
5. Create a Queue and demonstrate FIFO operations using offer(), peek(), and poll().
6. Create a Deque and demonstrate insertion, removal, and inspection at both ends.
7. Create Map, SortedMap, and NavigableMap objects and demonstrate key-value and navigation methods.
8. Use Iterator to traverse the collection forward.
9. Use ListIterator to traverse a List forward and backward.
10. Display the results on the console.

Interface Method Summary

Interface	Important methods
Collection	add(), addAll(), remove(), removeAll(), contains(), containsAll(), size(), isEmpty(), clear(), iterator()
List	add(), add(index, element), get(), set(), remove(index), indexOf(), lastIndexOf(), subList(), sort()
Set	add(), remove(), contains(), size(), isEmpty(), clear()
SortedSet	first(), last(), headSet(), tailSet(), subSet(), comparator()
NavigableSet	lower(), floor(), ceiling(), higher(), pollFirst(), pollLast(), descendingSet()
Queue	add(), offer(), remove(), poll(), element(), peek()
Deque	addFirst(), addLast(), offerFirst(), offerLast(), removeFirst(), removeLast(), pollFirst(), pollLast(), peekFirst(), peekLast()
Map	put(), get(), remove(), containsKey(), containsValue(), keySet(), values(), entrySet(), size(), isEmpty(), clear()
SortedMap	firstKey(), lastKey(), headMap(), tailMap(), subMap(), comparator()
NavigableMap	lowerKey(), floorKey(), ceilingKey(), higherKey(), firstEntry(), lastEntry(), pollFirstEntry(), pollLastEntry(), descendingMap()
Iterator	hasNext(), next(), remove(), forEachRemaining()
ListIterator	hasNext(), hasPrevious(), next(), previous(), remove(), set(), add()

Source Code – Eclipse Ready

```
package javacore;

import java.util.*;

public class CollectionMethodsDemo {

    public static void main(String[] args) {

        // 1. Collection / List
        List<String> list = new ArrayList<>();

        list.add("Java");
        list.add("Python");
        list.add("C");
        list.add("Java");

        System.out.println("LIST: " + list);
        System.out.println("Size: " + list.size());
        System.out.println("Contains Java: " + list.contains("Java"));
        System.out.println("Element at index 1: " + list.get(1));

        list.set(1, "C++");
        System.out.println("After set: " + list);

        list.remove("C");
        System.out.println("After remove: " + list);

        // 2. Set
        Set<Integer> set = new HashSet<>();

        set.add(10);
        set.add(20);
        set.add(30);
        set.add(20);

        System.out.println("\nSET: " + set);
        System.out.println("Contains 20: " + set.contains(20));
        System.out.println("Set Size: " + set.size());

        // 3. SortedSet
        SortedSet<Integer> sortedSet = new TreeSet<>();

        sortedSet.add(50);
        sortedSet.add(10);
        sortedSet.add(30);
        sortedSet.add(20);

        System.out.println("\nSORTED SET: " + sortedSet);
        System.out.println("First: " + sortedSet.first());
        System.out.println("Last: " + sortedSet.last());

        // 4. NavigableSet
        NavigableSet<Integer> navSet = new TreeSet<>(sortedSet);

        System.out.println("\nNAVIGABLE SET: " + navSet);
        System.out.println("Lower than 30: " + navSet.lower(30));
        System.out.println("Floor of 30: " + navSet.floor(30));
        System.out.println("Ceiling of 25: " + navSet.ceiling(25));
        System.out.println("Higher than 30: " + navSet.higher(30));

        // 5. Queue
        Queue<String> queue = new LinkedList<>();

        queue.offer("A");
        queue.offer("B");
        queue.offer("C");

        System.out.println("\nQUEUE: " + queue);
        System.out.println("Peek: " + queue.peek());
        System.out.println("Poll: " + queue.poll());
        System.out.println("Queue after poll: " + queue);

        // 6. Deque
```

```

Deque<Integer> deque = new ArrayDeque<>();

deque.addFirst(10);
deque.addLast(20);
deque.addFirst(5);

System.out.println("\nDEQUE: " + deque);
System.out.println("First: " + deque.peekFirst());
System.out.println("Last: " + deque.peekLast());

deque.removeFirst();
deque.removeLast();

System.out.println("Deque after removal: " + deque);

// 7. Map
Map<Integer, String> map = new HashMap<>();

map.put(101, "Neha");
map.put(102, "Anu");
map.put(103, "Ravi");

System.out.println("\nMAP: " + map);
System.out.println("Value of key 101: " + map.get(101));
System.out.println("Contains key 102: " + map.containsKey(102));
System.out.println("Contains value Ravi: " + map.containsValue("Ravi"));

System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());
System.out.println("Entries: " + map.entrySet());

// 8. SortedMap
SortedMap<Integer, String> sortedMap = new TreeMap<>();

sortedMap.put(3, "C");
sortedMap.put(1, "A");
sortedMap.put(2, "B");

System.out.println("\nSORTED MAP: " + sortedMap);
System.out.println("First Key: " + sortedMap.firstKey());
System.out.println("Last Key: " + sortedMap.lastKey());

// 9. NavigableMap
NavigableMap<Integer, String> navMap = new TreeMap<>(sortedMap);

System.out.println("\nNAVIGABLE MAP: " + navMap);
System.out.println("Lower Key than 2: " + navMap.lowerKey(2));
System.out.println("Floor Key of 2: " + navMap.floorKey(2));
System.out.println("Ceiling Key of 2: " + navMap.ceilingKey(2));
System.out.println("Higher Key than 2: " + navMap.higherKey(2));

// 10. Iterator
System.out.println("\nITERATOR:");

Iterator<String> iterator = list.iterator();

while (iterator.hasNext()) {
    System.out.println(iterator.next());
}

// 11. ListIterator
System.out.println("\nLIST ITERATOR - FORWARD:");

ListIterator<String> listIterator = list.listIterator();

while (listIterator.hasNext()) {
    System.out.println(listIterator.next());
}

System.out.println("\nLIST ITERATOR - BACKWARD:");

while (listIterator.hasPrevious()) {
    System.out.println(listIterator.previous());
}
}
}

```

Compilation & Execution

Save the source code exactly as:

CollectionMethodsDemo.java

Compile:

```
javac -d . CollectionMethodsDemo.java
```

Run:

```
java javacore.CollectionMethodsDemo
```

Expected Output – Main Results

```
LIST: [Java, Python, C, Java]
Size: 4
Contains Java: true
Element at index 1: Python
After set: [Java, C++, C, Java]
After remove: [Java, C++, Java]
```

```
SET: [10, 20, 30]
Contains 20: true
Set Size: 3
```

```
SORTED SET: [10, 20, 30, 50]
First: 10
Last: 50
```

```
NAVIGABLE SET: [10, 20, 30, 50]
Lower than 30: 20
Floor of 30: 30
Ceiling of 25: 30
Higher than 30: 50
```

```
QUEUE: [A, B, C]
Peek: A
Poll: A
Queue after poll: [B, C]
```

```
DEQUE: [5, 10, 20]
First: 5
Last: 20
Deque after removal: [10]
```

```
SORTED MAP: {1=A, 2=B, 3=C}
First Key: 1
Last Key: 3
```

```
NAVIGABLE MAP: {1=A, 2=B, 3=C}
Lower Key than 2: 1
Floor Key of 2: 2
Ceiling Key of 2: 2
Higher Key than 2: 3
```

```
ITERATOR:
Java
C++
Java
```

```
LIST ITERATOR - FORWARD:
Java
C++
Java
```

```
LIST ITERATOR - BACKWARD:
Java
C++
Java
```

Result

The Java Collection Framework was successfully studied and implemented using different collection interfaces and their important methods. The corrected program successfully demonstrates List, Set, SortedSet, NavigableSet, Queue, Deque, Map, SortedMap, NavigableMap, Iterator, and ListIterator operations.

Important: Eclipse Copy-Paste Version

Create a Java Class named **CollectionMethodsDemo** inside the **javacore** package. Ensure the package declaration matches the package folder. Do not paste the main() method outside the class declaration.